

Delta Care

Plataforma de ensino para uma faculdade de medicina, com um assistente de IA que responde só a partir do material que o professor publicou.

GRUPO E PROJETO

Delta Care

INTEGRANTES

Arthur Veiga Demétrio
Jean Rodrigues Matias
Matheus Marques De
Souza

ENTREGA

25 de setembro de 2026

REPOSITÓRIO

github.com/jeanrmatias/DeltaCare

11

histórias concluídas

128

verificações
automáticas

34

rotas de API

9

tabelas

01 Objetivo da sprint

03 As histórias

05 Incremento entregue

07 Dados e arquitetura

09 Sobre escala

11 Retrospectiva

02 Sprint backlog

04 Decisões de experiência

06 Qualidade

08 Fora de escopo e débito

10 Próxima sprint

12 Como executar

01 Objetivo da sprint

Transformar o retorno da revisão do produto em incremento entregue: corrigir o que estava quebrado, completar o que estava pela metade e eliminar os pontos em que a interface prometia algo que não acontecia.

A Sprint 1 entregou o fluxo de login e a identidade visual. A Sprint 2 entregou o núcleo do produto: turmas, materiais, autenticação por sessão e o assistente de IA com RAG. A Sprint 3 partiu de uma **revisão do produto em uso**, registrada no documento *Apontamentos e melhorias — Delta Care* de 23/09/2026, que apontou nove itens priorizados.

Isso muda a natureza da sprint: o backlog não veio de suposição sobre o que o usuário precisaria. Veio de alguém usando a plataforma e registrando onde ela falhou — o ciclo de feedback da disciplina, fechado uma vez.

Sobre a numeração. O catálogo interno de módulos (`frontend/modulos.js`) usa a numeração do backlog do Trello, em que "Sprint 3" nomeia o módulo de Atividades, que não faz parte desta entrega e aparece na seção 10. Aqui, "Sprint 3" é o ciclo de desenvolvimento descrito neste documento.

02 Sprint backlog

Onze histórias, todas concluídas. A prioridade segue o documento de apontamentos; as três últimas surgiram durante a sprint.

ID	HISTÓRIA	PRIORIDADE	SITUAÇÃO
US-01	Ser avisado do que acontece na minha turma	Alta	Concluída
US-02	Interromper a resposta da IA	Alta	Concluída
US-03	Identificar usuários por nome	Alta	Concluída
US-04	Publicar um material em várias turmas	Média	Concluída
US-05	Ver meu progresso na tela inicial	Média	Concluída

ID	HISTÓRIA	PRIORIDADE	SITUAÇÃO
US-06	Ler o material sem baixar	Média	Concluída
US-07	Filtrar a lista de materiais	Baixa	Concluída
US-08	Cadastrar uma turma inteira por planilha	Baixa	Concluída
US-09	Avisos dentro da identidade do produto	Adicional	Concluída
US-10	Ver meus dados e sair da conta	Adicional	Concluída
US-11	Reportar conteúdo inadequado	Adicional	Catálogo corrigido

US-01 Notificações

ALTA

CONCLUÍDA

Como **professor**, quero ser avisado quando algo relevante acontecer na minha turma, para não precisar abrir a plataforma todos os dias para conferir.

O que havia: um `<span class="badge">5</span>` escrito à mão no HTML e nenhum tratador de clique. O "5" nunca significou nada, e o painel não abria.

- ✓ O contador mostra o número real de notificações não lidas.
- ✓ O clique abre o painel e lista os avisos.
- ✓ Clicar num aviso o marca como lido e o contador diminui.
- ✓ Avisos vêm de três eventos reais: aluno matriculado (avisa o professor), material publicado (avisa os alunos) e material agendado que chegou na data.
- ✓ Ninguém consegue marcar como lida a notificação de outra pessoa.

Decisão de projeto. O material agendado fica visível sozinho quando a data passa, sem nenhum código rodar naquele instante. Em vez de uma tarefa periódica só para emitir esse aviso, a verificação acontece quando o usuário abre o sino: custa uma consulta e o aviso chega no momento em que ele olharia de qualquer forma. A coluna `materiais.notificado` impede avisar duas vezes.

Segurança. `marcar_como_lida` filtra por `user_id` além do id da notificação — sem isso, conhecer o id bastaria para mexer na caixa de outro usuário. Há teste cobrindo exatamente essa tentativa.

Como **aluno**, quero interromper a geração de uma resposta, para não ficar preso esperando algo que já não me serve.

O modelo roda localmente e leva de 10 a 20 segundos. Antes, a única saída era recarregar a página.

- ✓ O botão "Parar" **substitui** o "Enviar" durante a geração.
- ✓ Cancelar aborta a requisição de `AbortController`), não só esconde o fato (resultado.
- ✓ O cancelamento mostra "Resposta interrompida", texto diferente do erro de rede.
- ✓ A interface volta ao estado normal e o campo recebe o foco.

Por que substituir e não somar um botão: enviar outra pergunta enquanto a primeira é gerada não é uma ação válida. Um botão que o usuário não pode usar agora não deve estar disponível agora.

Como **administração**, quero registrar o nome de quem cadastro, para identificar pessoas por nome e não por endereço de e-mail.

- ✓ Nome é obrigatório na criação de conta, nos três perfis.
- ✓ Professor tem campo de disciplinas; aluno tem matrícula.
- ✓ Contas que já existiam, sem nome, continuam funcionando.
- ✓ O nome aparece na saudação e no rodapé da barra lateral.

Decisão de migração. A coluna `nome` aceita `NULL` de propósito: declará-la `NOT NULL` quebraria as contas existentes. A obrigatoriedade vive na criação da conta; para conta antiga, a interface deriva um nome legível do e-mail ("ana.paula@..." vira "Ana Paula"). A migração é automática na primeira execução e não perde dados.

Além do pedido, o formulário já matricula o aluno numa turma no momento da criação, evitando cadastrar todos e depois matricular um a um.

Como **professor**, quero publicar o mesmo material em várias turmas de uma vez, para não repetir o upload turma por turma.

- ✓ Lista de turmas do professor com checkbox e "Selecionar todas", que só aparece quando há mais de uma turma.
- ✓ Se qualquer turma selecionada não for do professor, **nada** é criado.
- ✓ O arquivo é gravado uma única vez, mesmo indo para cinco turmas.
- ✓ Excluir o material de uma turma não apaga o arquivo enquanto outra turma ainda o referenciar.

Decisão de modelagem. `turma_id` fica na linha do material, então publicar em N turmas cria N registros. Migrar para muitos-para-muitos quebraria a listagem por turma do professor e a busca por turma do assistente, sem ganho real — o professor edita e despublica turma a turma. Mas como o arquivo é compartilhado, `excluir_material` conta referências antes de apagar do disco; sem isso, excluir de uma turma deixaria as outras apontando para um arquivo que não existe mais. Um PDF de 15 MB em cinco turmas ocuparia 75 MB à toa.

A validação acontece antes de criar qualquer registro: publicação parcial deixaria o professor sem saber em quais turmas o material entrou.

Como **aluno**, quero ver meu progresso ao entrar, para saber onde estou sem procurar numa aba de desempenho.

- ✓ Nível, barra de XP, sequência de dias e gráfico dos últimos 14 dias.
- ✓ A composição do XP é visível na tela.
- ✓ Nenhum número é estimado.

A restrição que definiu o desenho. O módulo de Atividades não existe, então não há nota nem entrega para pontuar. O XP vem exclusivamente do que o sistema observa de fato:

AÇÃO REGISTRADA	XP
Pergunta ao assistente	10
Material distinto consultado	15
Dia com atividade	25

Para que "material consultado" existisse, foi criada a tabela `acessos_material`, gravada quando o aluno abre ou baixa um material — e **só depois** da checagem de permissão, para que material que ele não pode ver não entre no acompanhamento dele.

Mostrar a composição não é enfeite: um total de pontos sem origem explicada não engaja, irrita. Quando Atividades existir, entrega e nota entram na mesma conta sem refazer a tela.

Dois detalhes de comportamento: reabrir o mesmo PDF cinco vezes conta como um material (`COUNT(DISTINCT)`), e a sequência de dias começa a contar de ontem quando hoje ainda não teve atividade — senão zeraria toda manhã.

US-06 Ler o material sem baixar

MÉDIA

CONCLUÍDA

Como **aluno**, quero abrir o material dentro da plataforma, para não acumular arquivos baixados só para estudar.

- ✓ PDF, texto, imagem, vídeo e áudio abrem num modal.
- ✓ O arquivo é buscado autenticado; o token não aparece em URL.
- ✓ Formatos que o navegador não renderiza não oferecem "Visualizar".

Por que blob e não um iframe apontando para a rota do arquivo: a navegação do iframe não envia o header `Authorization`, e passar o token na URL o deixaria gravado no histórico do navegador e nos logs do servidor. O arquivo é buscado autenticado, vira um blob local e é exibido a partir dele — liberado ao fechar, senão cada abertura deixaria o arquivo inteiro na memória.

Por que .docx e .xlsx não têm o botão: prometer visualização e entregar uma tela cinza é pior do que dizer que não dá. Para esses, resta o download.

US-07 Filtros na lista de materiais

BAIXA

CONCLUÍDA

Como **aluno**, quero filtrar os materiais, para achar o da aula de hoje sem percorrer a lista inteira.

- ✓ Filtros combinados de assunto, tópico, tipo e período (7, 30 ou 90 dias), somados à busca por texto que já existia.
- ✓ Botão para limpar tudo.
- ✓ As opções vêm dos materiais que o aluno realmente tem.

Uma lista fixa de assuntos ofereceria filtros que não retornam nada e esconderia assuntos que o professor cadastrou.

Como **administração**, quero cadastrar uma turma inteira a partir da planilha que já tenho, para não digitar aluno por aluno.

- ✓ Aceita CSV e XLSX.
- ✓ "Conferir planilha" valida **sem gravar nada** ; só depois "Importar" cria as contas.
- ✓ Relatório linha a linha do que entrou e do que foi recusado, com o motivo.
- ✓ Linha inválida é recusada individualmente; as demais entram.

Duas etapas de propósito. Uma planilha com metade das linhas erradas não deve criar metade das contas para o administrador descobrir depois.

Sem dependência nova. O `.xlsx` é um ZIP com XML dentro, e `zipfile` + `ElementTree` são da biblioteca padrão. Trazer pandas ou openpyxl para ler três colunas seria desproporcional; o `requirements.txt` segue com três pacotes.

Tolerância a planilha do mundo real: separador `;` (Excel em português) ou `,`; codificação UTF-8, UTF-8 com BOM, cp1252 ou latin-1; cabeçalho em várias grafias ("Nome Completo", "Aluno", "E-mail", "Matrícula", "RA", "Registro").

Importar de PDF foi avaliado e recusado. O documento pedia para avaliar a viabilidade. Texto extraído de PDF não tem estrutura de tabela, e adivinhar coluna por posição no papel erra em silêncio. Num cadastro acadêmico, um aluno com a matrícula de outro é pior do que pedir a planilha.

Como **usuário de qualquer perfil**, quero que avisos e confirmações apareçam dentro da identidade do produto, para saber que é o sistema falando comigo.

Vinte e um `alert()`, `confirm()` e `prompt()` do navegador foram substituídos.

✓ Nenhuma chamada nativa de `alert`, `confirm` ou `prompt` no código.

✓ Fecha com Esc, o foco fica preso dentro do diálogo e leitores de tela o anunciam como modal — o

`<dialog>` nativo entrega isso.

✓ Todos retornam Promise, então o código chamador ficou praticamente igual.

Os diálogos nativos travam a página inteira, ignoram a identidade visual e, em alguns navegadores, **podem ser silenciados pelo usuário** — o que faria uma confirmação de exclusão desaparecer sem aviso.

Ganho colateral: a recuperação de senha pedia e-mail, código e nova senha em três `prompt()` seguidos, e quem errasse o último perdia o código. Agora código e senha ficam no mesmo diálogo.

US-10 Ver meus dados e sair

ADICIONAL

CONCLUÍDA

Como **usuário de qualquer perfil**, quero ver os dados da minha conta e sair dela pelo mesmo lugar onde meu nome aparece.

O rodapé da barra lateral trazia um "Ver perfil" que era **texto solto**, sem botão nem tratador: clicar não fazia nada. E não havia lugar óbvio para sair.

✓ "Ver perfil" abre um diálogo com os dados reais da conta, pela rota `/eu` .

✓ "Sair" encerra a sessão no servidor, não só no navegador.

O perfil é **só leitura**. Editar o próprio cadastro é item de escopo futuro — mas mostrar o que já existe é outra coisa, e é o que faz o menu deixar de ser um beco sem saída.

US-11 Denúncias: uma lacuna no próprio roadmap

ADICIONAL

CATÁLOGO CORRIGIDO

Esta história não corrigiu código: corrigiu o **planejamento**.

Revisando o catálogo de módulos, apareceu que o módulo de denúncias existia só do lado de quem trata. A administração tinha "receber e listar denúncias", "acompanhar o status" e "registrar a ação tomada". Mas não havia, em lugar nenhum, como uma denúncia **ser criada** — nem pelo aluno, nem pelo professor. Uma caixa de entrada sem porta de entrada.

A lacuna vinha do backlog original: os cards descrevem a gestão das denúncias e nenhum descreve o ato de denunciar. É o efeito de escrever backlog por tela em vez de por fluxo — cada tela parece completa sozinha.

Correção: o catálogo ganhou o módulo "Denúncias" nas áreas do aluno e do professor, descrevendo o lado de quem reporta, e os dois módulos passaram a se referenciar explicitamente — cada um declara que depende do outro e que os dois são a mesma entrega. O item entrou no menu dos dois perfis, para não virar uma página que ninguém alcança.

04 Decisões de experiência

A disciplina trata experiência como uma coisa só: do usuário final, de quem opera e de quem mantém. As decisões abaixo foram tomadas nessa chave.

Declarar o que não existe, em vez de simular

Os módulos ainda não construídos não são botões mortos nem telas com dados de exemplo: cada item do menu leva a uma página que diz o que aquele módulo vai fazer e em qual sprint entra — são 10 módulos catalogados em `frontend/modulos.js`. O roadmap fica visível dentro do próprio produto, e nada na tela finge estar pronto.

Nenhum número inventado

Foi por essa regra que o XP nasceu de acessos e perguntas registrados, e não de uma estimativa plausível. Um indicador que o usuário descobre ser fictício destrói a confiança em todos os outros.

Um botão só existe quando a ação é possível

"Parar" substitui "Enviar"; "Visualizar" não aparece em formato que o navegador não renderiza; "Selecionar todas" não aparece com uma turma só.

Dizer o que aconteceu, com a palavra certa

Cancelamento é "Resposta interrompida", não erro. A importação devolve linha a linha o motivo de cada recusa, em vez de "planilha inválida".

Consistência por token, não por repetição

Cores, tipografia e espaçamento vivem em `frontend/tokens.css` e são consumidos por variável CSS — 278 referências nas três folhas de estilo. Trocar a cor primária da instituição é editar um lugar.

Acessibilidade: o que está feito e o que não está

Feito: `<dialog>` nativo com foco preso, Esc e semântica de modal; `role="alert"` na mensagem de login; 15 campos com `<label for>`; 9 controles só de ícone com `aria-label`; e o Markdown da IA renderizado sem `innerHTML`, de modo que HTML vindo do modelo não é interpretado.

Não feito: auditoria de contraste e navegação completa por teclado testada com leitor de tela. Está registrado como débito, não como pronto.

05 Incremento entregue

Estado da plataforma ao fim da sprint. Em negrito, o que esta sprint acrescentou.

Aluno

Tela inicial com **nível, XP, sequência e gráfico de 14 dias**; lista de materiais com busca e **filtros de assunto, tópico, tipo e período**; **visualizador interno** de PDF, texto, imagem, vídeo e áudio; assistente de IA restrito ao material da sua turma, com histórico e **botão Parar**; **sino de notificações**; **perfil e sair**.

Professor

Painel da turma; materiais com rascunho, agendamento e publicação, **em várias turmas de uma vez**; **visualizador interno**; **notificações reais**; **perfil e sair**.

Administração

Turmas, professores e matrículas; criação de conta de qualquer perfil **com nome, disciplinas e matrícula**, e **matrícula já na criação**; **importação por CSV/XLSX em duas etapas com relatório**; gestão de usuários.

Transversal

Diálogos próprios no lugar dos nativos; catálogo de módulos futuros; política de privacidade.

06 Qualidade

```
python backend/testes.py           # 80 testes (não precisa do Ollama)
python frontend/testar_html.py      # 14 páginas

cd frontend && node testes.mjs       # 34 testes (roda dentro de frontend/)
```

Tudo verde na entrega: 80, 34 e 14 sem nenhuma falha. Eram 43 testes de backend antes da sprint e nenhum de JavaScript.

O que os testes cobrem, e por quê. O foco é o que dá prejuízo se quebrar em silêncio: permissão, visibilidade de material, sessão e integridade do banco ao excluir. Interface e formatação ficam de fora de propósito — erro de CSS aparece na tela, erro de permissão não aparece nunca.

- Aluno não vê material de turma em que não está matriculado.
- Rascunho e agendado não vazam para o aluno.
- Ninguém marca como lida a notificação de outro.
- O arquivo compartilhado entre turmas só é apagado quando o último registro que o referencia é excluído.

Os testes de backend rodam sem o Ollama. As duas únicas chamadas ao modelo são recebidas como parâmetro pelas demais funções, então o teste injeta uma função falsa. É também o que permitiria trocar de provedor de IA sem mexer no resto.

`testar_html.py` nasceu de um erro nosso, contado na retrospectiva: usa o parser da biblioteca padrão, não expressão regular, porque contar tags não detecta aninhamento errado.

07 Dados e arquitetura

Nove tabelas: `users`, `turmas`, `materiais`, `matriculas`, `material_chunks`, `chat_mensagens`, `sessoes`, `notificacoes` e `acessos_material` — as três últimas criadas nesta sprint, junto da coluna `materiais.notificado`.

Autenticação por token opaco de sessão, guardado na tabela `sessoes` com validade de 12 horas. Não é JWT: token no banco pode ser revogado na hora e não exige gerenciar chave de assinatura. JWT faria sentido com vários serviços validando sem consultar o banco, que não é o caso de um deploy por instituição.

Identidade nunca vem do cliente. As dependências `usuario_logado` e `exigir_perfil` são o único caminho pelo qual uma rota descobre quem está chamando. Nenhuma rota protegida aceita e-mail vindo do corpo da requisição para dizer quem é o usuário.

O backend separa o que o sistema usa do que o sistema decide: `infra/` guarda banco, hash de senha, sessão e gravação de arquivo; `regras/` guarda autenticação, turmas, materiais, visão do aluno, matrículas, notificações, importação e o RAG.

A visão do aluno é um arquivo separado da visão do professor, mesmo consultando as mesmas tabelas. Na visão do professor toda consulta inclui rascunho e agendado; na do aluno, nunca. Unificar as duas numa função com um parâmetro convidaria a um erro de filtro que vazaria material não liberado.

08 Fora de escopo e débito técnico

Fora de escopo por decisão registrada

- **Autosserviço de perfil** — o próprio documento de apontamentos pediu para registrar como futuro e não implementar. O perfil ficou só leitura.
- **Importar de PDF** — avaliado e recusado, com a justificativa na US-08.

Débito técnico conhecido

O que separa isto de um produto em produção:

- Sem HTTPS: o token viaja em texto claro. Aceitável em rede local de demonstração, obrigatório antes de qualquer dado real.
- Sem limite de tentativas no login e na recuperação de senha.
- SQLite, que não lida bem com concorrência; Postgres é o caminho.
- Arquivos em disco local, não em storage de nuvem.
- Recuperação de senha imprime o código no console, sem serviço de e-mail.
- A indexação do PDF roda dentro da requisição de upload, um embedding por trecho: com PDF grande, a requisição fica presa por minutos. Precisa virar trabalho em segundo plano.
- A busca vetorial carrega todos os trechos da turma e calcula similaridade em Python a cada pergunta. Com material acumulado, precisa de `sqlite-vec` ou `pgvector`.

- Acessibilidade: contraste não auditado, navegação por teclado não testada com leitor de tela.

09 Sobre escala

O assistente roda em **Ollama local**, com `gpt-oss:20b` para chat e `nomic-embed-text` para embeddings. Duas consequências medidas:

- A GPU de desenvolvimento, de 12 GB, não comporta o modelo inteiro: cerca de 10,5 GB cabem na VRAM e o resto vai para a CPU, o que explica os 13 a 20 segundos por resposta. Não há configuração que feche essa diferença.
- O Ollama serializa requisições. Para vários alunos ao mesmo tempo, o caminho é vLLM ou SGLang, que fazem lotes contínuos.

Por isso o produto é **um deploy por instituição**. Uma faculdade tem uma GPU e um pico de uso previsível; SaaS multi-inquilino teria de resolver isolamento de dados, fila global de inferência e custo por aluno ao mesmo tempo. O recorte atual é o que o grupo consegue entregar funcionando de verdade, e é defensável em vez de ser uma limitação escondida.

Duas medições definiram a busca do assistente. Recuperar 5 trechos por pergunta perdia informação em 8% dos casos, e 10 recuperou todos. E só similaridade de cosseno não bastava: num trecho de 800 caracteres em que apenas 80 respondem à pergunta, o embedding é dominado pelo resto, e o trecho certo caía para a 7ª posição. Um bônus pequeno para trechos que contêm literalmente os termos distintivos da pergunta — siglas, códigos e números, que é o que identifica assunto em texto médico — levou-o à 1ª sem piorar nenhuma outra pergunta, e perguntas fora do material continuaram sendo recusadas.

10 Próxima sprint

Na ordem em que o produto pede:

- **Atividades**, para aluno e professor. É o que falta para o XP deixar de ser só engajamento e virar desempenho: entrega e nota entram na mesma conta já construída na US-05.

- **Denúncias**, os dois lados juntos, conforme a US-11. Implementar só a gestão repetiria a lacuna.
- **Mensagens professor e aluno**, para o que o assistente de IA não resolve.
- **Calendário e relatórios da coordenação**.

Antes de implementar qualquer um deles: conferir se outros módulos do catálogo têm a mesma assimetria da US-11, com um lado especificado e o outro não. A lacuna das denúncias não foi um descuido isolado, foi consequência de como o backlog foi escrito.

11 Retrospectiva

Funcionou

Partir de uma revisão do produto em uso

Os nove itens eram problemas reais, e um deles — as notificações — era uma funcionalidade que parecia existir e não existia.

Escrever o porquê junto do código

As decisões de modelagem e segurança ficaram registradas, o que tornou possível retomar o trabalho sem reconstruir o raciocínio.

Testar a regra, não a tela

Os 80 testes de backend pegam erros de permissão que nenhuma inspeção visual pegaria.

Não funcionou, e o que aprendemos

Edição em massa de HTML por script quebrou as páginas duas vezes

O script calculava posições de texto e aplicava substituições de tamanho diferente, cortando no meio de uma tag — numa das vezes engoliu `</aside>`, `<main>` e o cabeçalho de doze páginas. Recuperamos do git. Edição estrutural de marcação precisa de parser, e o que nos deu confiança falsa foi a verificação por expressão regular, que conta tags e não entende aninhamento. Daí nasceu o `testar_html.py`.

Um teste que replicava a lógica de produção passava com o código quebrado

O teste de diálogos tinha uma cópia da normalização de campos; quando a de produção quebrava, a do teste não. Extraímos a função para os dois usarem a mesma e confirmamos que o teste falha quando o erro é reintroduzido. Um teste que não falha nunca é decoração.

Um teste frágil escondia um problema real

Conferir que o catálogo tinha exatamente 9 módulos quebrava a cada módulo novo e não verificava nada útil. Foi substituído por duas verificações de coerência — e foi uma delas que documentou a lacuna das denúncias.

Cache do servidor de desenvolvimento nos fez depurar código que não estava rodando

O servidor embutido do Python não envia `Cache-Control`, e o navegador serviu JavaScript antigo. Passamos a usar `frontend/servir.py`, com `no-store`, e a versionar os assets.

O que levamos para a próxima

- Backlog escrito por **fluxo**, não por tela. É o que teria evitado a US-11.
- Nenhuma verificação nova entra como "ok" sem que a gente veja ela falhar primeiro, com o erro reintroduzido de propósito.

12 Como executar

```
pip install -r requirements.txt
python backend/seed_demo.py                # dados de demonstração

cd backend  && python -m uvicorn main:app --port 8000
cd frontend  && python servir.py              # em outro terminal
```

Depois, abrir `http://localhost:5500`. As contas de demonstração são impressas pelo `seed_demo.py`. O assistente de IA exige o Ollama rodando com `gpt-oss:20b` e `nomic-embed-text`; o resto da plataforma funciona sem ele.

